DaVinci Configurator Classic AutomationInterface

Development Getting Started of the AutomationInterface (AI)

DaVinci Configurator Classic Team

March 23, 2023

© 2023
Vector Informatik GmbH
Ingersheimerstr. 24
70499 Stuttgart

Contents

1 Introduction **3**
- 1.1 General . . . 3
- 1.2 Facts . . . 3

2 Getting started with Script Development **5**
- 2.1 General . . . 5
- 2.2 Automation Script Development Types . . . 5
- 2.3 Script File . . . 5
- 2.4 Script Project . . . 7
  - 2.4.1 Script Project Development . . . 10
  - 2.4.2 Java JDK Setup . . . 10
  - 2.4.3 IntelliJ IDEA Setup . . . 10
  - 2.4.4 Gradle Setup . . . 11

3 AutomationInterface Architecture **12**
- 3.1 Components . . . 12
- 3.2 Languages . . . 13
  - 3.2.1 Why Groovy . . . 13
- 3.3 Script Structure . . . 14
  - 3.3.1 Scripts . . . 14
  - 3.3.2 Script Tasks . . . 15
  - 3.3.3 Script Locations . . . 15
- 3.4 Script loading . . . 15
  - 3.4.1 Internal Script Reload Behavior . . . 15
- 3.5 Script editing . . . 16
- 3.6 Licensing . . . 16
- 3.7 Script Coding Conventions and Constraints . . . 16
  - 3.7.1 Usage of static fields . . . 17
  - 3.7.2 Usage of Outer Closure Scope Variables . . . 18
  - 3.7.3 States over script task execution . . . 18
  - 3.7.4 Usage of Threads . . . 18
  - 3.7.5 Usage of DaVinci Configurator Classic private Classes Methods or Fields 18

1 Introduction

1.1 General

The user of the DaVinci Configurator Classic can create scripts, which will be executed inside of the Configurator to:

- Create projects
- Update projects
- Manipulate the data model with an access to the whole AUTOSAR model
- Generate code
- Executed repetitive tasks with code, without user interaction
- More

The scripts are written by the *user* with the DaVinci Configurator Classic AutomationInterface.

1.2 Facts

Installation The DaVinci Configurator Classic can execute customer defined scripts out of the box. No additional scripting language installation is required by the customer.

Languages The scripts are written in Groovy or Java. See 3.2 on page 13 for details.

Debugging Support The scripts can be debugged via IntelliJ IDEA. See chapter "Automation Script Project Debugging" in the Automation Interface Documentation.

Documentation The AutomationInterface provides a comprehensive documentation:

- This document
- Javadoc HTML pages as class reference
- Script samples and templates
  - ScriptProject creation assistant in the DaVinci Configurator Classic
- API documentation inside of an IDE
- Integrated Definition (BSWMD) description for all modules in the SIP

Code Completion You have code completion for Groovy and Java for the DaVinci Configurator Classic AutomationInterface. You have to use IntelliJ IDEA for code completion.[1]

[1]See chapter "Automation Script Project" in the Automation Interface Documentation

There is also a SIP based code completion for contained Module, Container and Parameter definitions. This eases the traversal through the AUTOSAR model.

© 2023, Vector Informatik GmbH

2 Getting started with Script Development

2.1 General

This chapter gives a short introduction of how to get started with script file or script project creation.

Attention: You need at least one of the **License Options .WF or .MD** to develop scripts. The script project creation assistant will not be available otherwise. Please note that the execution of a script requires no specific license.

2.2 Automation Script Development Types

The DaVinci Configurator Classic supports two types of automation scripts:

- Script files (`.dvgroovy` files)
- Script projects (`.jar` files)

Script File The script file provides the **simplest way** to implement an automation script. When the script gets bigger you should migrate to a script project.

To create a script file proceed with chapter 2.3.

Script Project The script project is **more effort** to create and maintain, but provides IDE support for:

- Code completion
- Syntax highlighting
- API Documentation
- Debug support
- Build support

It is the **recommended way to develop** scripts, containing more tasks or multiple classes.

To create a script project proceed with chapter 2.4 on page 7.

2.3 Script File

The script file is the simplest way to implement an automation script. It could be sufficient for small tasks and if the developer does not need support by the tool during implementing the script and if debugging is not required.

Prerequisites Before you start, please make sure that you have a **SIP** containing a DaVinci Configurator Classic 5 available on your system.

Creation Inside your SIP you find examples of automation script files. Create your own script folder and copy an example, e.g. `...ScriptSamples/SimpleScript.dvgroovy` to your folder.

Rename the script file and open it in any text editor. In case of `SimpleScript.dvgroovy` it consists of several tasks. One of the tasks will print a "HelloApplication" string to the console.

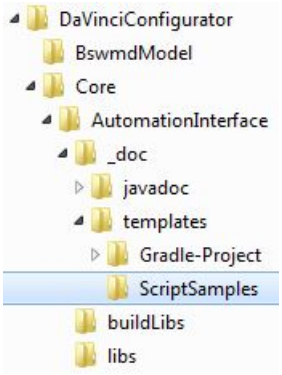


Figure 2.1: Script Samples location

Open the DaVinci Configurator Classic inside your SIP. If not yet visible open the Views

- Script Locations
- Script Tasks

via the View menu.

In the **Script Locations** View select the location folder User@Machine. On its context menu you can **Add** a script location. Select your own script folder.



Figure 2.2: Script Locations View

Alternatively you could add the script location to the Session folder. In this case the script location would only be stored in the current session.

Switch to the **Script Tasks** View. It provides an overview over the tasks contained in your script.

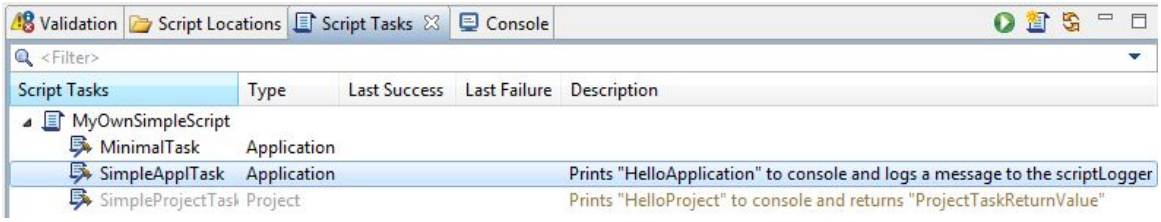


Figure 2.3: Script Tasks View

Execute the SimpleAppTask by double-click or by the Execute Command contained in its context menu or by the Execute Button of the Task View and check that "HelloApplication" is printed in the console.

You can modify the implementation according to your needs. For the AutomationInterface API Reference see chapter “Automation Interface API Reference” in the Automation Interface Documentation. It is sufficient to edit and save the modifications in your editor. The file is automatically reloaded by the DaVinci Configurator Classic then and can be executed immediately.

Debugging It is not possible to debug a script file, if you want to debug, please migrate to a script project, see chapter 2.4.

2.4 Script Project

The script project is the preferred way to develop an automation script, if the content is more than one simple task.

A script project is a normal IDE project (IntelliJ IDEA recommended), with compile bindings to the DaVinci Configurator Classic AutomationInterface. It is also called "Automation Script Project" throughout this document.

The DaVinci Configurator Classic will load a script project as a single `.jar` file. So the script project must be built and packaged into a `.jar` file before it can be executed by the DaVinci Configurator Classic.

Prerequisites Before you start, **please make sure** that the following items are available on your system:

- **SIP** containing a DaVinci Configurator Classic 5
- **Java JDK**: For the development with the IntelliJ IDEA a "Java SE Development Kit 8" (JDK 8) is required. Please install the JDK 8 as described in chapter 2.4.2 on page 10.
- **IDE**: For the script project development the *recommended* IDE is *IntelliJ IDEA*. Please install IntelliJ IDEA as described in chapter 2.4.3 on page 10.
- **Build system**: To build the script project the build system Gradle is required. See chapter 2.4.4 on page 11 for installation instructions.

Project Creation Open the DaVinci Configurator Classic inside your SIP. If not yet visible open the following Views via the View menu:

- Script Locations
- Script Tasks

Switch to the View **Script Tasks** and select the Button **Create New Script Project...**.



Figure 2.4: Create New Script Project... Button

© 2023, Vector Informatik GmbH

Note: If the button is not available, please make sure you have least one of the **License Options .WF or .MD** to develop scripts.

The **New Automation Script Project** dialogs is opened. Click *Next* because you are reading the document.

On the second page first you have to select a Script template on which the new project shall be based on. Please select `Default Automation Project` and click *Next*.

On the third page **Project Settings**, please specify the following items:

- **Script Project Name**
  - Define a name for your new project.
- **Project Location**
  - Select a parent folder in which your project shall be created in.
    Note: A new folder with the project name is created in this folder.
- **Gradle Distribution URL**
  - Select one option:
    - **Gradle Default**
      - This will download the required Gradle build system. To use this option you need **internet access**.
    - **Custom URL**
      - Specify an URL to your own Gradle distribution.
        New settings are displayed to specify the path. To setup your own Gradle build system see 2.4.4 on page 11.
- **Open IntelliJ IDEA**
  - Select this option if the project shall automatically be opened in IntelliJ IDEA after creation. In case IntelliJ IDEA is not installed on your system a warning will be issued.

© 2023, Vector Informatik GmbH

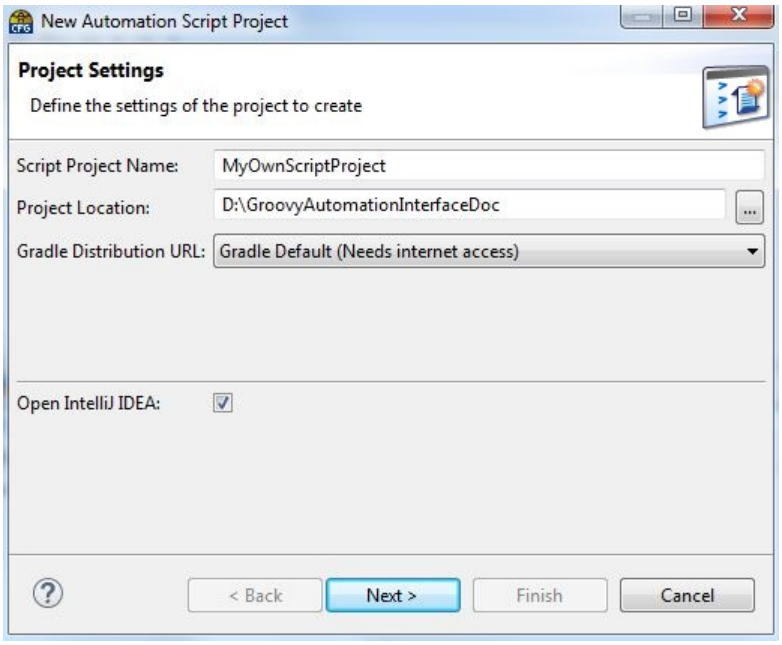


Figure 2.5: Project Settings

Proceed until the dialog is finished.

A new project will be created. Necessary tasks as setting up the IntelliJ IDEA and building the project are automatically initiated. At the end IntelliJ IDEA will be started with the created project.

You can now modify the implementation according to your needs. For the AutomationInterface API Reference see chapter “Automation Interface API Reference” in the Automation Interface Documentation. To edit and rebuild the project use IntelliJ IDEA.

After each build the project is automatically reloaded by the DaVinci Configurator Classic and can be executed there.

IntelliJ IDEA Usage Ensure that the Gradle JVM and the Project SDK are set in the IntelliJ IDEA Settings. For details see 2.4.3 on the following page.

Having modified and saved `MyScript.groovy` in the IntelliJ IDEA editor you can build the project by pressing the **Run Button provided in the toolbar**. The functionality of this Run Button is determined by the option selected in the Menu beneath this button. In this menu **<ProjectName> [build] shall be selected**.



Figure 2.6: Project Build

For more information to IntelliJ IDEA usage please see chapter “IntelliJ IDEA Usage” in the Automation Interface Documentation. If you have trouble with IntelliJ, see chapter “IntelliJ IDEA Troubleshooting” in the Automation Interface Documentation.

Debugging To debug the script project follow the instructions in chapter “Automation Script Project Debugging” in the Automation Interface Documentation.

DaVinci Configurator Classic views The View **Script Tasks** provides an overview over the scripts and tasks contained in the project. The newly created project already contains a sample script file `MyScript.groovy`.

The `Default Automation Project` sample script file contains one task that prints a "Hello-Application" string to the console. Run and check it as already described in 2.3 on page 5. If you have selected a different Script Sample the `MyScript.groovy` will contain the sample code.

The View **Script Locations** contains the path to the script project build folder containing the built `.jar` file.

Jar Location The Jar location of the built script project is `<ProjectDir>/build/libs`. Gradle will automatically create the directories during the build and will generate the built `.jar` file.

2.4.1 Script Project Development

For more details to the development of a script project see chapter “Automation Script Project” in the Automation Interface Documentation.

2.4.2 Java JDK Setup

Install a JDK 8 on your system. The Java JDK website provides download versions for different systems. Download an appropriate version.

The DaVinci Tool Suite only supports 64 Bits, so make sure you get the x64 version.

The JDK is needed for the Java Compiler for IntelliJ IDEA and Gradle.

2.4.3 IntelliJ IDEA Setup

Install IntelliJ IDEA on your system. The IntelliJ IDEA website provides download versions for different applications. Download[1] a version that supports Java and Groovy and that is in the list of supported versions (see list “IntelliJ IDEA Versions” in the Automation Interface Documentation).

Code completion and compilation additionally require that the Project SDK is set. Therefore open the File -> **Project Structure** Dialog in IntelliJ IDEA and switch to the settings dialog for **Project**. If not already available set an appropriate option for the **Project SDK**. Please set the value to a valid Java JDK (see 2.4.2). **Do not** select a JRE.

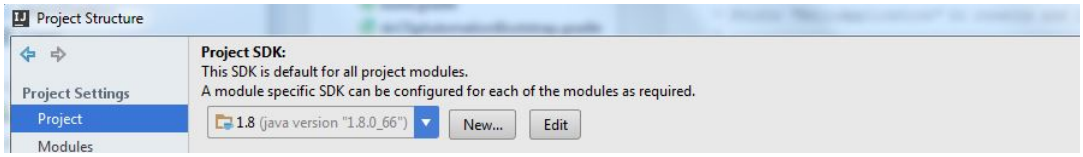


Figure 2.7: Project SDK Setting

[1] Vector-Internal: If you are inside of the Vector intranet, you could download it from: `file:////vistrpesfs1/project2/DaVinci/Eclipse/Platform/CFG5/BuildComponents/IntelliJ`

To enable building of projects ensure that the Gradle JVM is set. Therefore open the File -> **Settings** Dialog in IntelliJ IDEA and find the settings dialog for **Gradle**. If not already available set an appropriate option for the **Gradle JVM**. Please set the value to the same Java JDK as the Project SDK above. **Do not** select a JRE.

If you do not have the Gradle settings, please make sure that the Gradle plugin inside of IntelliJ IDEA is installed. Open the File -> **Settings** Dialog then Plugins and select the Gradle plugin.

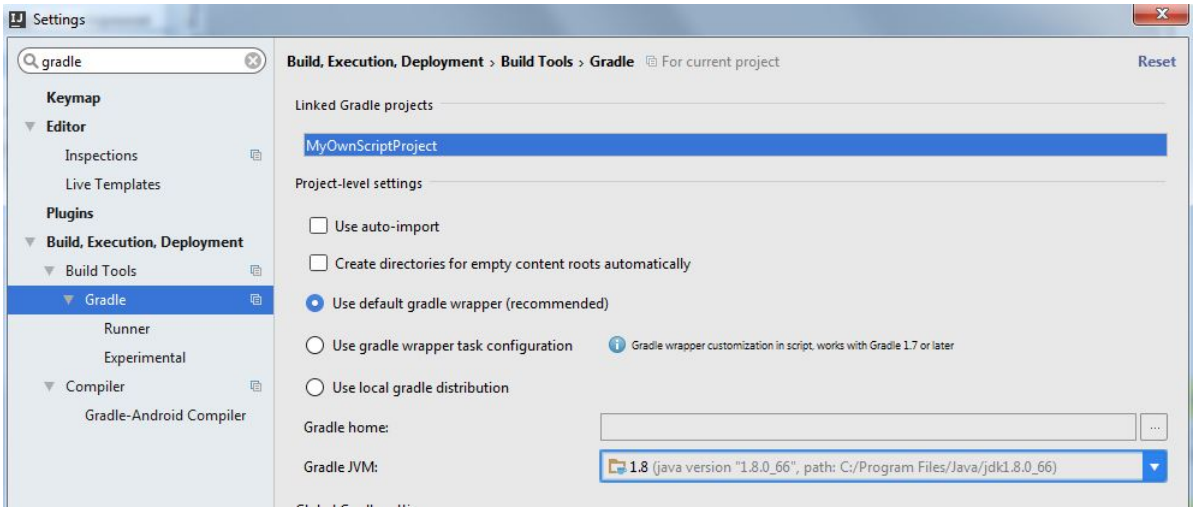


Figure 2.8: Gradle JVM Setting

2.4.4 Gradle Setup

If your system has internet access you can use the default Gradle Build System provided by the DaVinci Configurator Classic. In this case you **do not** have to install Gradle. If you are a Vector internal user you could also **skip** the Gradle installation.

If you want to use your own Gradle Build System install it on your system. The Gradle website provides the required download version for the Gradle Build System. Please **download the version 7.3.3**. See chapter “Build System” in the Automation Interface Documentation for more details to the Build System.

3 AutomationInterface Architecture

3.1 Components

The DaVinci Configurator Classic consists of three components:

- Core components
- AutomationInterface (AI) - also called Automation API
- Scripting engine

The other part is the script provided by the user.

The Scripting engine will load the script, and the script uses the AutomationInterface to perform tasks. The AutomationInterface will translate the requests from the script into Core components calls.

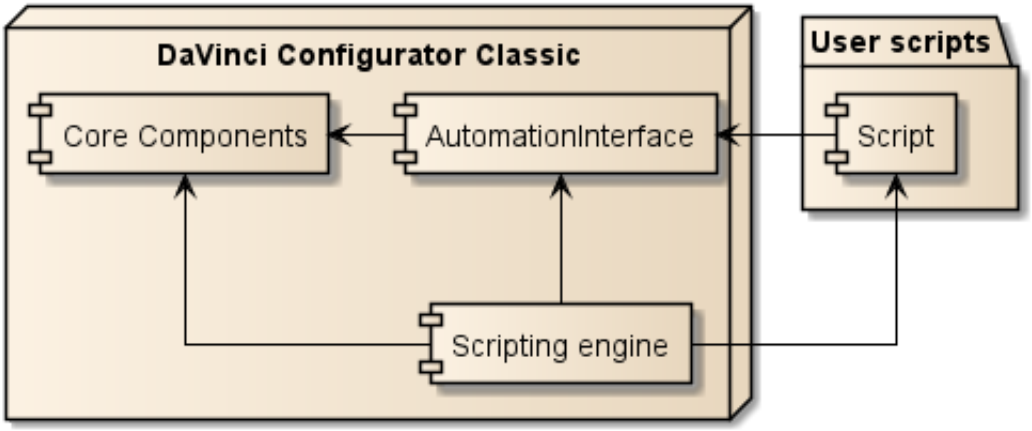


Figure 3.1: DaVinci Configurator Classic components and interaction with scripts

The separation of the AutomationInterface and the Core components has multiple benefits:

- Stable API for script writters
  - Including checks, that the API will not break in following releases
- Well defined and documented API
- Abstraction from the internal heavy lifting
  - This ease the usage for the user, because the automation interfaces are tailored to the use cases.

PublishedApi All AutomationInterface classes are marked with a special annotation to **highlight** the fact that it is part of the published API. The annotation is called `@PublishedApi`.

So every class marked with `@PublishedApi` can be used by the client code. But if a class is **not** marked with `@PublishedApi` or is marked with `@Deprecated` it should not be used by any client code, nor shall a client call methods via reflection or other runtime techniques.

You should **not** access DaVinci Configurator Classic private or package private classes, methods or fields.

3.2 Languages

The DaVinci Configurator Classic provides out of the box language support for:

- Java[1]
- Groovy[2]

The recommended scripting language is **Groovy** which shall be preferred by all users.

3.2.1 Why Groovy

Flat Learning Curve Groovy is concise, readable with an expressive syntax and is easy to learn for Java developers[3].

- Groovy syntax is 95%-compatible with Java[4]
- Any Java developer will be able to code in Groovy without having to know nor understand the subtleties of this language

This is very important for teams where there's not much time for learning a new language.

Domain-Specific Languages (DSL) Groovy has a flexible and malleable syntax, advanced integration and customization mechanisms, to integrate readable business rules in your applications.

The DSL features of Groovy are extensively used in DaVinci Automation API to provide simple and expressive syntax.

Powerful Features The Groovy language supports Closures, builders, runtime & compile-time meta-programming, functional programming, type inference, and static compilation.

Website The website of Groovy is http://groovy-lang.org. It provides a good documentation and starting guides for the Groovy language.

Groovy Book The book **"Groovy in Action, Second Edition"**[5] provides a comprehensive guide to Groovy programming language. It is written by the developers of Groovy.

[1] `http://http://www.java.com` [2016-05-09]

[2] `http://groovy-lang.org` [2016-05-09]

[3] Copied from `http://groovy-lang.org` [2016-05-09]

[4] Copied from `http://melix.github.io/blog/2010/07/27/experience_feedback_on_groovy.html` [2016-05-09]

[5] Groovy in Action, Second Edition by Dierk König, Paul King, Guillaume Laforge, Hamlet D'Arcy, Cédric Champeau, Erik Pragt, and Jon Skeet June 2015 ISBN 9781935182443
`https://www.manning.com/books/groovy-in-action-second-edition` [2016-05-09]

© 2023, Vector Informatik GmbH

3.3 Script Structure

A script always contains one or more script tasks. A script is represented by an instance of `IScript`, the contained tasks are instances of `IScriptTask`.

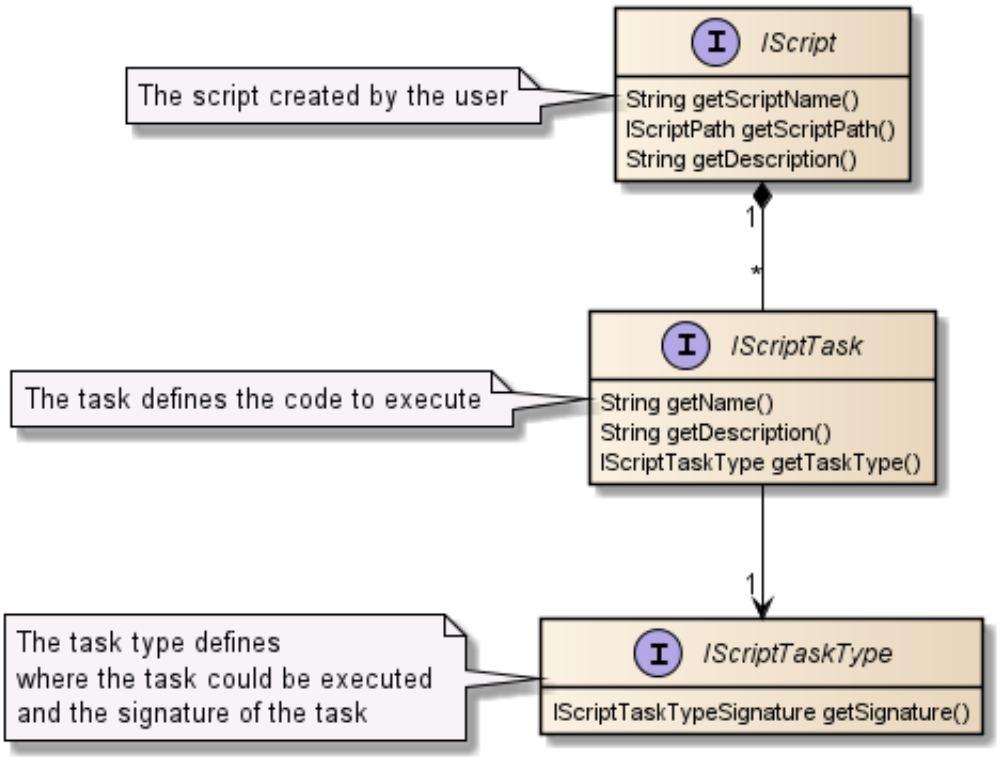


Figure 3.2: Structure of scripts and script tasks

You create the `IScript` and `IScriptTask` instance with the API described in chapter "Script API" in the Automation Interface Documentation.

The script task type (`IScriptTaskType`) defines where the task could be executed. It also defines the signature of the task's `code {}` block. See chapter "Script Task Types" in the Automation Interface Documentation for the available script task types.

3.3.1 Scripts

Script contain the tasks to execute and are loaded from the script locations specified in the DaVinci Configurator Classic.

The DaVinci Configurator Classic supports two types of automation scripts:

- Script files (`.dvgroovy` files)
- Script projects (`.jar` files)

For details to the script project, see chapter "Automation Script Project" in the Automation Interface Documentation.

© 2023, Vector Informatik GmbH

3.3.2 Script Tasks

Script tasks are the executable units of scripts, which are executed at certain points in the DaVinci Configurator Classic (specified by the `IScriptTaskType`). Every script task has a `code {}` block, which contains the logic to execute.

3.3.3 Script Locations

Script locations define where script files are loaded from. These locations are edited in the DaVinci Configurator Classic Script Locations view. You can also start the Configurator with the option `-scriptLocations` to specify additional locations.

The DaVinci Configurator Classic could load scripts from different script locations:

- SIP
- Project
- User-defined directories
- More

3.4 Script loading

All scripts contained in the script locations are automatically loaded by the DaVinci Configurator Classic. If new scripts are added to script locations these scripts are automatically loaded.

If a script changes during runtime of the DaVinci Configurator Classic the whole script is reloaded and then executable, without a restart of the tool or a reload of the project.

This enables script development during the runtime of the DaVinci Configurator Classic

- No project reload
- No tool restart
- Faster feedback loops

Note: A jar file of a script project *should be updated by the Gradle build system*, not by hand. Because the Java VM is holding a lock to the file. If you try to replace the file in the explorer you will get an error message.

3.4.1 Internal Script Reload Behavior

Your script can be loaded and unloaded automatically multiple times during the execution of the DaVinci Configurator Classic. More precise, when a script is currently not used and there are memory constraints your script will be automatically unloaded.

If the script will be executed again, it is automatically reloaded and then executed. So it is possible that the script initialization code is called multiple times in the DaVinci Configurator Classic lifecycle. But this is no issue, because the script and the tasks **shall not** have any internal state during initialization.

© 2023, Vector Informatik GmbH

Memory Leak Prevention The feature above is implemented to prevent leaking memory from an automation script into the DaVinci Configurator Classic memory. So when the memory run low, all unused scripts are unloaded, which will also free leaked memory of scripts.

But this **does not** mean that is impossible to construct memory leaks from an automation script. E.g. Open file handles without closing them will still cause a memory leak.

3.5 Script editing

The DaVinci Configurator Classic does not contain any editing support for scripts, like:

- Script editor
- Debugger
- REPL (Read-Eval-Print-Loop)

These tasks are delegated to other development tools:

- IntelliJ IDEA (recommended)
- EclipseIDE
- Notepad++

See chapter “Automation Script Project” in the Automation Interface Documentation for script development and debugging with IntelliJ IDEA.

3.6 Licensing

The DaVinci Configurator Classic requires certain license options to develop and/or execute script tasks.

You need specific combinations of DaVinci Configurator Classic licenses and options to develop and execute scripts.

For typical automation script tasks you need following licenses:

- Product license CFG PRO required for execution of script tasks
- Additional option .WF required for development and debugging

For generation script tasks for example you need a different license combination.

See chapter “Script Task Types” in the Automation Interface Documentation for details, which script task type requires which license.

Some script task may require different licenses or options during development or execution. It is also possible that the execution does not require any license at all. Normally you need more license options to develop scripts than you need to execute them.

3.7 Script Coding Conventions and Constraints

This section describes conventions, which you are advised to apply.

© 2023, Vector Informatik GmbH

Requirement Levels - Wording

- Shall: This word, or the terms "Mandatory", "Required" or "Must", mean that the rule or convention is an absolute requirement.
- Shall not: This word, or the terms "Must not" mean that the rule or convention is an absolute prohibition.
- Should: This word, or the adjective "Recommended", mean that there may exist valid reasons in particular circumstances to ignore a particular item, but the full implications must be understood and carefully weighed before choosing a different course.
- Should not: This phrase, or the phrase "Not recommended" mean that there may exist valid reasons in particular circumstances when the particular behavior is acceptable or even useful, but the full implications should be understood and the case carefully weighed before implementing any behavior described with this label.
- May: This word, or the adjective "Optional", mean that an item is truly optional.

See also "RFC 2119: Key words for use in RFCs to Indicate Requirement Levels"[6].

3.7.1 Usage of static fields

You **shall not** use any static fields in your script code or other written classes inside of your project. Except `static final` constants of simple immutable types like (normally compile time constants):

- int
- boolean
- double
- String
- ...

Static fields will cause memory leaks, because the fields are not garbage collected. Example:

```
scriptTask("Name"){
  code{
    MyClass.leakVariable.add("Leaked Memory")
  }

}

class MyClass{
  static List leakVariable = []
}
```

Listing 3.1: Static field memory leak

The use of static fields of the AutomationInterface is allowed.

[6] https://www.ietf.org/rfc/rfc2119.txt

3.7.2 Usage of Outer Closure Scope Variables

The same static field rule applies to variables passed from outer `Closure` scopes into a script task `code{}` block. You **shall not** cache/save data into such variables.

Example:

```
scriptTask("Name"){
 def invalidVariable = [] //List

  code{
    invalidVariable.add("Leaked Memory")
  }

}
```

Listing 3.2: Memory leak with closure variable

3.7.3 States over script task execution

You **shall not** hold or save any states over multiple script task executions in your classes.

The script task should be state less. All states are provided by the Automation API or the data models.

If you need to cache data over multiple executions, see chapter “Stateful Script Tasks” in the Automation Interface Documentation for a solution.

3.7.4 Usage of Threads

A script task **shall not** create any `Thread, Executor, ThreadPool or ForkJoinPool` instances. If multithreading is required, the Automation API provides the corresponding methods.

A different thread will not provide any Automation APIs and will cause IllegalStateExceptions.

3.7.5 Usage of DaVinci Configurator Classic private Classes Methods or Fields

A script task **should not** call or rely on any non published API or private (also package private) classes, methods or fields. You also should not use any reflection techniques to reflect about Configurator internal APIs. Otherwise it is not guaranteed that your script will work with other DaVinci Configurator Classic versions. See 3.1 on page 12 for details about PublishedApi.

But it is valid to use reflection for your own script code.

© 2023, Vector Informatik GmbH